PRACTICAL - 09

Aim: Counter race (Counter.java + Driver.java)

Code Screenshot:

Practical_09 - Visual Studio Code — File Edit Selection View Go Run Terminal Help

Counter.java — Java

lab-09 > src > Counter.java

```java
public class Counter {
    private int count = 0;

    public void incrementUnsafe() {
        count++;
    }

    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}
```

Driver.java — Java

lab-09 > src > Driver.java

```java
import java.util.concurrent.CountDownLatch;

public class Driver {
    private static final int THREADS = 8;
    private static final int INCREMENTS = 250_000;

    private static void runTest(Counter counter, boolean safe)
            throws InterruptedException {
        CountDownLatch start = new CountDownLatch(1);
        Thread[] threads = new Thread[THREADS];

        for (int i = 0; i < THREADS; i++) {
            threads[i] = new Thread(() -> {
                try {
                    start.await();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
                for (int j = 0; j < INCREMENTS; j++) {
                    if (safe) {
                        counter.increment();
                    } else {
                        counter.incrementUnsafe();
                    }
                }
            });
            threads[i].start();
        }

        start.countDown();
        for (Thread thread : threads) {
            thread.join();
        }
    }

    public static void main(String[] args) throws InterruptedException {
        int expected = THREADS * INCREMENTS;

        Counter unsafeCounter = new Counter();
        runTest(unsafeCounter, false);

        Counter safeCounter = new Counter();
        runTest(safeCounter, true);

        System.out.println("Expected count: " + expected);
        System.out.println("Without synchronization: " + unsafeCounter.getCount());
        System.out.println("With synchronization: " + safeCounter.getCount());
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood race conditions and critical section vulnerabilities in Java.
- Mastered the synchronized method modifier for mutual exclusion.
- Coordinated thread completion deterministically using Thread.join().
- Analyzed memory visibility and read-modify-write atomicity guarantees.
- Verified thread safety across concurrent multi-threaded execution paths.

PRACTICAL - 09

Aim: Seat booking race (CinemaBooking.java + BookingDriver.java)

Code Screenshot:

Practical_09 - Visual Studio Code — File Edit Selection View Go Run Terminal Help

CinemaBooking.java — Java

lab-09 > src > CinemaBooking.java

```java
import java.util.concurrent.CountDownLatch;

public class CinemaBooking {
    private int seatsLeft = 5;
    private final CountDownLatch allChecked;

    public CinemaBooking(int buyers) {
        allChecked = new CountDownLatch(buyers);
    }

    public boolean bookUnsafe() throws InterruptedException {
        if (seatsLeft > 0) {
            allChecked.countDown();
            allChecked.await();
            seatsLeft--;
            return true;
        }
        return false;
    }

    public synchronized boolean book() {
        if (seatsLeft > 0) {
            seatsLeft--;
            return true;
        }
        return false;
    }

    public int getSeatsLeft() { return seatsLeft; }
}
```

BookingDriver.java — Java

lab-09 > src > BookingDriver.java

```java
import java.util.concurrent.atomic.AtomicInteger;

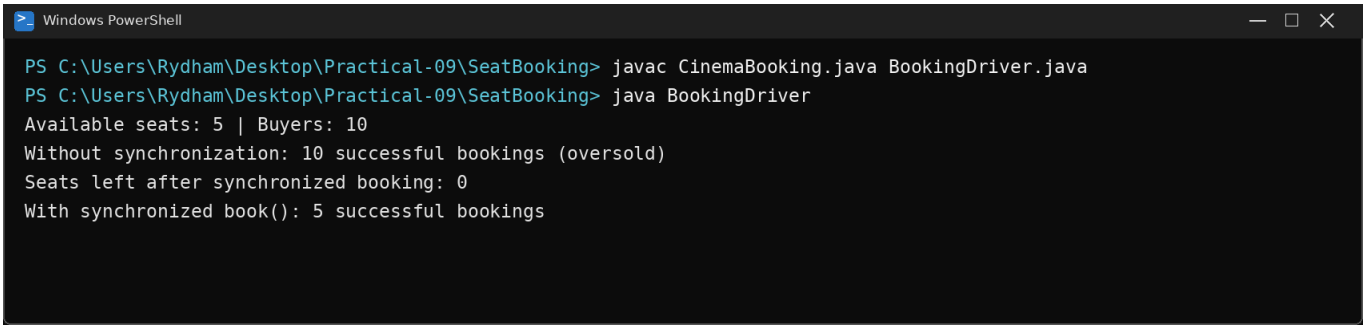
public class BookingDriver {
    private static final int BUYERS = 10;

    private static int run(boolean safe) throws InterruptedException {
        CinemaBooking cinema = new CinemaBooking(safe ? 0 : BUYERS);
        AtomicInteger booked = new AtomicInteger();
        Thread[] threads = new Thread[BUYERS];

        for (int i = 0; i < BUYERS; i++) {
            threads[i] = new Thread(() -> {
                try {
                    boolean success = safe ? cinema.book() : cinema.bookUnsafe();
                    if (success) booked.incrementAndGet();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
            threads[i].start();
        }
        for (Thread thread : threads) thread.join();
        if (safe) System.out.println("Seats left after synchronized booking: "
                + cinema.getSeatsLeft());
        return booked.get();
    }

    public static void main(String[] args) throws InterruptedException {
        System.out.println("Available seats: 5 | Buyers: 10");
        System.out.println("Without synchronization: " + run(false)
                + " successful bookings (oversold)");
        System.out.println("With synchronized book(): " + run(true)
                + " successful bookings");
    }
}
```

Output Screenshot:



Learning Outcome:

- Protected limited shared resources against concurrent overselling.
- Applied monitor locks to guarantee serialized access to critical sections.
- Analyzed the behavior of competing concurrent threads under contention.
- Maintained data integrity in ticket-reservation architectures.
- Implemented safe state updates within synchronized transactional boundaries.

PRACTICAL - 09

Aim: Parallel array sum race (ArraySum.java + ArraySumDriver.java)

Code Screenshot:

Practical_09 - Visual Studio Code — File Edit Selection View Go Run Terminal Help

ArraySum.java — Java

lab-09 > src > ArraySum.java

```java
import java.util.concurrent.atomic.AtomicLong;

public class ArraySum {
    private long total;

    public void addUnsafe(long value) {
        total += value;
    }

    public synchronized void addSynchronized(long value) {
        total += value;
    }

    public long getTotal() {
        return total;
    }

    public static long calculate(long[] numbers, int mode)
            throws InterruptedException {
        ArraySum sum = new ArraySum();
        AtomicLong atomic = new AtomicLong();
        Thread[] workers = new Thread[8];
        int chunk = numbers.length / workers.length;

        for (int i = 0; i < workers.length; i++) {
            final int start = i * chunk;
            final int end = (i == workers.length - 1)
                    ? numbers.length : start + chunk;
            workers[i] = new Thread(() -> {
                for (int j = start; j < end; j++) {
                    if (mode == 0) {
                        sum.addUnsafe(numbers[j]);
                        if ((j & 1023) == 0) Thread.yield();
                    } else if (mode == 1) {
                        sum.addSynchronized(numbers[j]);
                    } else {
                        atomic.addAndGet(numbers[j]);
                    }
                }
            });
            workers[i].start();
        }

        for (Thread worker : workers) {
            worker.join();
        }
        return mode == 2 ? atomic.get() : sum.getTotal();
    }
}
```

ArraySumDriver.java — Java

lab-09 > src > ArraySumDriver.java

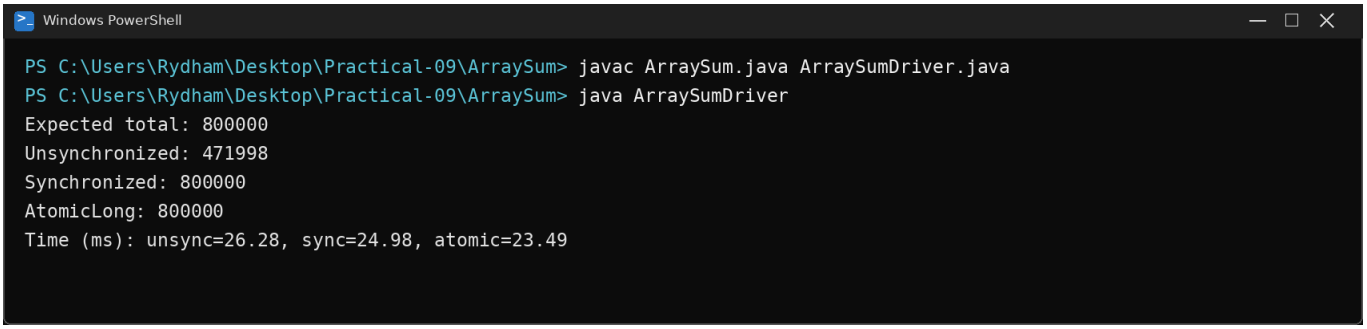
```java
public class ArraySumDriver {
    private static final int SIZE = 800_000;

    public static void main(String[] args) throws InterruptedException {
        long[] numbers = new long[SIZE];
        for (int i = 0; i < SIZE; i++) {
            numbers[i] = 1;
        }

        long[] results = new long[3];
        double[] times = new double[3];
        for (int mode = 0; mode < 3; mode++) {
            long start = System.nanoTime();
            results[mode] = ArraySum.calculate(numbers, mode);
            times[mode] = (System.nanoTime() - start) / 1_000_000.0;
        }

        System.out.println("Expected total: " + SIZE);
        System.out.println("Unsynchronized: " + results[0]);
        System.out.println("Synchronized: " + results[1]);
        System.out.println("AtomicLong: " + results[2]);
        System.out.printf("Time (ms): unsync=%.2f, sync=%.2f, atomic=%.2f%n",
                times[0], times[1], times[2]);
    }
}
```

Output Screenshot:



Learning Outcome:

- Divided a large array into independent thread workloads.
- Observed lost updates when threads shared an unsynchronized total.
- Used synchronized methods to protect shared accumulator updates.
- Applied AtomicLong for thread-safe concurrent addition.
- Compared execution time using System.nanoTime().